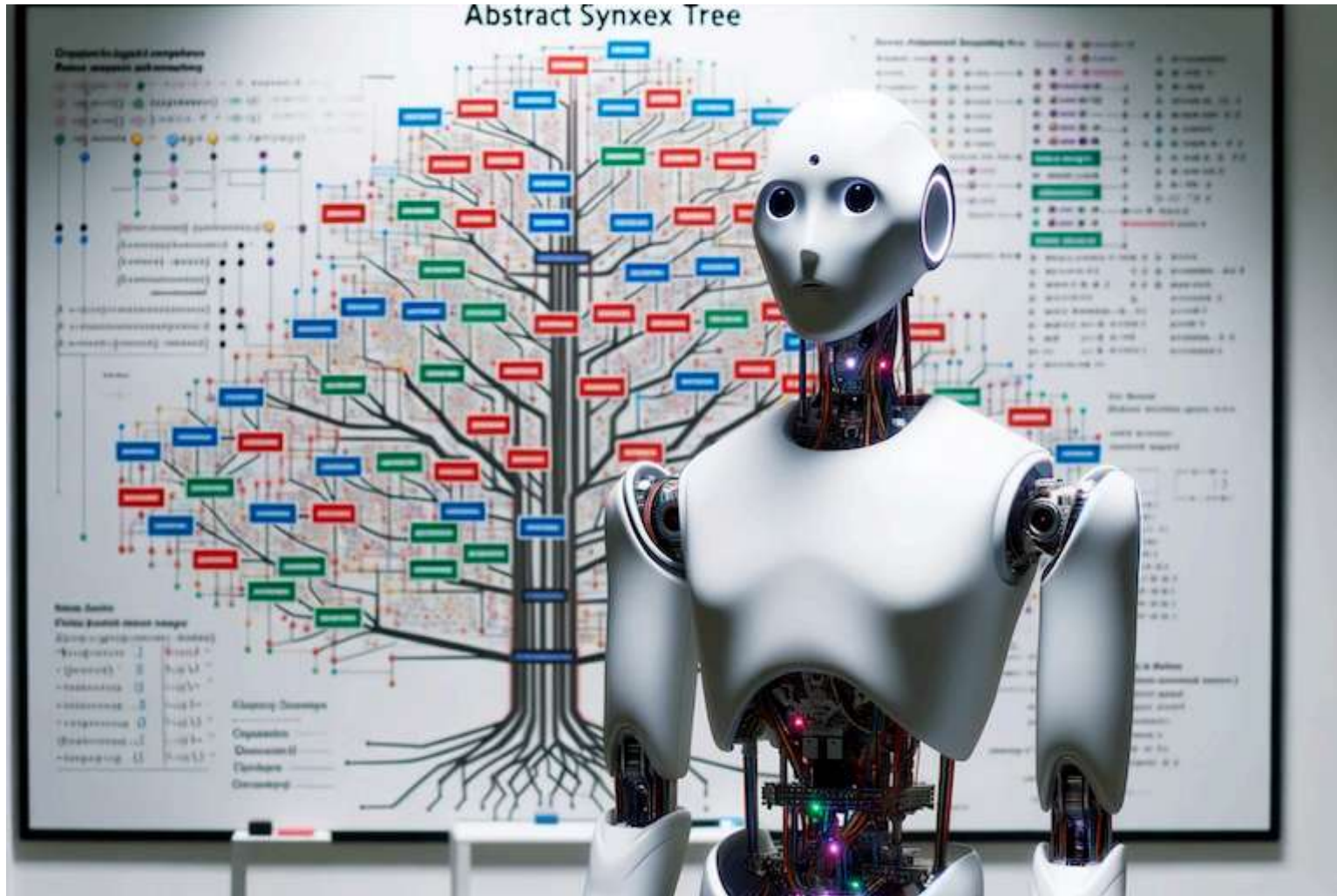


Repository map



Aider uses a **concise map of your whole git repository** that includes the most important classes and functions along with their types and call signatures. This helps aider understand the code it's editing and how it relates to the other parts of the codebase.

The repo map also helps aider write new code that respects and utilizes existing libraries, modules and abstractions found elsewhere in the codebase.

Using a repo map to provide context

Aider sends a **repo map** to the LLM along with each change request from the user. The repo map contains a list of the files in the repo, along with the key symbols which are defined in each file. It shows how each of these symbols are defined, by including the critical lines of code for each definition.

Here's a part of the repo map of aider's repo, for [base_coder.py](#) and [commands.py](#) :

```
aider/coders/base_coder.py:
:...
| class Coder:
|     abs_fnames = None
:...
|     @classmethod
|     def create(
|         self,
|         main_model,
|         edit_format,
|         io,
|         skip_model_availability_check=False,
|         **kwargs,
:...
|     def abs_root_path(self, path):
:...
|     def run(self, with_message=None):
:...
|
```

```
aider/commands.py:
:...
| class Commands:
|     voice = None
|
:...
|     def get_commands(self):
```

```
:\...
|   def get_command_completions(self, cmd_name, partial):
:\...
|   def run(self, inp):
:\...
```

Mapping out the repo like this provides some key benefits:

- The LLM can see classes, methods and function signatures from everywhere in the repo. This alone may give it enough context to solve many tasks. For example, it can probably figure out how to use the API exported from a module just based on the details shown in the map.
- If it needs to see more code, the LLM can use the map to figure out which files it needs to look at. The LLM can ask to see these specific files, and aider will offer to add them to the chat context.

Optimizing the map

Of course, for large repositories even just the repo map might be too large for the LLM's context window. Aider solves this problem by sending just the **most relevant** portions of the repo map. It does this by analyzing the full repo map using a graph ranking algorithm, computed on a graph where each source file is a node and edges connect files which have dependencies. Aider optimizes the repo map by selecting the most important parts of the codebase which will fit into the active token budget. The optimization identifies and maps the portions of the code base which are most relevant to the current state of the chat.

The token budget is influenced by the `--map-tokens` switch, which defaults to 1k tokens. Aider adjusts the size of the repo map dynamically based on the state of the chat. It will usually stay within that setting's value. But it does expand the repo map significantly at times, especially when no files have been added to the chat and aider needs to understand the entire repo as best as possible.

The sample map shown above doesn't contain *every* class, method and function from those files. It only includes the most important identifiers, the ones which are most often referenced by other portions of the code. These are the key pieces of context that the LLM needs to know to understand the overall codebase.

More info



Please check the [repo map article on aider's blog](#) for more information on aider's repository map and how it is constructed.
